

A Physical and Timing Aware Placement Optimization Framework Based on Graph Neural Network

Wenjie Ding^{1,2}, Zhanhua Zhang^{1,2}, Guoqing He¹ and Peng Cao^{1,2*}

¹National ASIC System Engineering Technology Research Center, Southeast University, Nanjing, China

²National Center of Technology Innovation for EDA, Nanjing, China
caopeng@seu.edu.cn

Abstract

Timing-driven placement is crucial in physical design flow with significant impact on later routability and ultimate manufacturability, which may deviate from finding the optimal solution and/or lead to unnecessary iterations, suffering from interleaved optimization steps and the corresponding inaccurate timing estimation. To solve this issue, we propose a **Physical and Timing Aware** framework with **Graph Neural Network**, **PTA-GNN**, which provides the candidate gate sizing and buffer insertion solutions as well as the timing constraint for potential violated paths as guidance to improve placement quality significantly. Experimental results on the *OpenCores* benchmarks with 22nm technology demonstrate that the proposed placement optimization framework achieves up to 89.09% worst negative slack (WNS), 55.47% total negative slack (TNS) improvement and 25.36% reduction on the number of violating paths (#VP). Our framework benefits the later routing stage with 2.19% wirelength decrease and 22% runtime reduction compared to standard physical design flow.

Keywords

Placement optimization, timing closure, buffer insertion, graph neural networks

1 Introduction

Circuit placement is at the heart of physical implementation of modern VLSI design, whose solution quality would pose significant impact on later routability and ultimate manufacturability [8]. During finding the optimal locations for logic cells at both global and detail placement stages, interleaved optimization steps including gate sizing and buffering are performed to achieve timing closure [24]. As the result, formidable challenges are introduced to existing heuristic or analytic placement methods from both **physical** and **timing** perspectives, leading to sub-optimal optimization solutions and increased turnaround time [10, 15].

From **physical** perspective, gate sizing and buffer insertion steps during placement optimization would inevitably cause profound

interference to the estimation of total wirelength, cell density and other approximations of interconnect cost for placement. Unfortunately, the lack of netlist reconstruction information and updated pin capacitance information results in flow-wise sub-optimal solution for the NP-hard combinational problems of cell placement [12, 27, 30]. Although the machine learning (ML) approaches were recently proposed to predict or generate the sizing and buffering solutions at placement stage [11, 23], few of them integrate the predicted sizing and/or buffering solutions into modern design flow to guide placement optimization behavior with high quality and efficiency.

From **timing** perspective, timing-driven placement is preferred to yield immediate performance benefits, focusing on only total wirelength with equal attention to all nets [15]. On one hand, without considering the influence of interleaved optimization steps, the estimation accuracy of circuit timing would be dramatically degraded so that the placement may be misled [4]. On the other hand, by targeting the optimization of the most critical path as commercial tools do, more iterations for placement optimization are required for the left sub-critical paths with timing violations, suffering from runtime cost increase [28]. In spite of recent ML-based approaches for accurate cross-stage timing prediction [1, 7, 9], the application of improving the placement performance has not been extensively studied yet.



Figure 1: Proposed framework in physical design flow.

To take the above issues into full consideration, we propose a universal placement optimization framework that provides comprehensive **Physical and Timing Aware** guidance generated by **Graph Neural Network** (GNN), namely **PTA-GNN**, to improve the quality and efficiency of detail placement, as shown in Figure 1. The goal of this work is to present a generalizable placement optimization framework that can improve end-of-flow performance target metrics. Our key contributions are summarized as follows.

- To the best of our knowledge, we are the first work to present physical and timing aware guidance for placement optimization owing to an integrated graph learning framework, which achieves significant placement quality improvement and runtime speedup.
- The candidate gate sizing and buffer insertion solutions are predicted as physical aware guidance with GNNs, where a *matrixized buffer tree generation mechanism* is introduced to represent buffer trees with multiple fanouts.

This work was supported in part by the National Natural Science Foundation of China under Grant 62174031 and in part by the Fundamental Research Funds for the Central Universities.

*Peng Cao is the corresponding author.



This work is licensed under a Creative Commons Attribution International 4.0 License.
ICCAD '24, October 27–31, 2024, New York, NY, USA
© 2024 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-1077-3/24/10.
<https://doi.org/10.1145/3676536.3676772>

- *Multi-embedding fusion* technique is employed in the GNN for the estimation of potential violated path slack to improve accuracy by fusing the embeddings of gate sizing and buffering predictions, which is provided as timing aware guidance for placement optimization.
- We conduct experiments and ablation studies on benchmarks in the advanced 22-nm technology nodes to confirm the effectiveness of our proposed optimization framework in commercial tools.

The rest of the paper is organized as follows. In Section 2, learning-based related works and motivation are introduced. Section 3 explains how to utilize our proposed *PTA-GNN* model to achieve prediction and optimization. Section 4 reports our experimental results. Section 5 concludes the paper and outlines future research directions.

2 Related Works and Motivation

2.1 Learning-based Timing Prediction

As a primary design metric, timing needs to be considered early in physical design flow. A fast and accurate timing prediction can essentially benefit design quality by providing early and high-fidelity feedback to timing-driven placement optimization. Extensive researches have been triggered for early timing prediction with learning engines including linear regression [1], random forest (RF) [1, 9, 11], XGboost [6], GNN [7, 31] and Transformer [2, 23].

However, as reported in [30], during timing optimization, on average 40%/21% nets/cells are replaced and an average of 59.6%/33.3% net/cell delay change is induced accordingly for those unreplaced cells, which prevents prior ML-based methods from accurately predicting the timing metric and hinders the predictor from guiding physical design with high quality.

2.2 Learning-based Circuit Optimization

Prior learning-based optimization approaches mainly contain generative ML approaches [13, 14, 21], reinforcement learning optimization [16, 17] and learning-model-guided electronic design automation (EDA) flow [24] that struggled to perform tool-accurate optimization with significant efficiency improvement or provide optimization solutions with better performance, power, area (PPA) metrics. The generative ML approach directly generates an optimized netlist for gate sizing [23] and buffer insertion [14] by utilizing Transformer to deliver similar PPA metrics as commercial tools with tremendous runtime speedup but has difficulty to be integrated into EDA tools or algorithms and hardly exceed the performance of the commercial tool that is learned from. As a promising ML paradigm to achieve super-human performance, reinforcement learning (RL) is applied to improve the quality of placement with gate sizing [17] but may suffer from notorious sample inefficiency, long training time and limited generalization [26].

Recently, some related works based on ML models demonstrate great potential to optimize commercial physical design flow by leveraging model guidance in the areas of design rule check (DRC) amelioration [3], path based analysis (PBA)-driven optimization [22], timing constraints improvement [24], cell clustering by unsupervised learning [5, 19, 20] and other techniques [4, 25]. In order to overcome the miscorrelation between layout maps of different

stages, Andrew B. Kahng *et al.* [3] proposed an engine that minimally perturbs the converged physical synthesis netlist to effectively predict detailed-route DRC violations after global routing, and then guide detailed-route to reduce DRC violations without hurting timing, area or wirelength. To improve Quality of Results (QoR) at physical design stage, Synopsys Inc. [22] uses machine-learning models to predict PBA timing based on graph based analysis (GBA) and guide post-route optimization in a commercial EDA tool flow, resulting in improved area/power without compromising on signoff timing closure. In [24], the authors introduce EaSyOpt to embed generic logic paths into a low-dimensional space of representation, making use of predicted critical paths information to guide the optimization algorithms. The concept of placement guidance is presented in the works of [5, 19, 20] to provide designers placement optimization frameworks that achieve effective and high-quality placement by distilling underlying design knowledge, where the cell clustering solutions are predicted in an unsupervised manner and provided to commercial placers as soft constraints to improve final PPA metrics.

Although prior approaches provide productive candidate solutions for circuit optimization, the corresponding timing impact due to netlist change and/or layout change may be not considered thoroughly, resulting in the degradation of Placement and Routing (PnR) quality improvement when being recommended to design flows. How to provide effective optimization solutions at early stage with timing insight for better PPA quality and less runtime iteration remains uninvestigated in existing researches.

2.3 Graph Neural Network

GNN has been widely employed in EDA field for various design tasks [18, 30] to perform effective graph representation learning. As a variant of GNN, graph attention network (GAT) introduces a self-attention mechanism. When taking the neighbors into account, it is rational to give more attention to the nodes similar to the node of interest.

The input GAT layer is a set of node features, $\mathbf{h} = \{\vec{h}_1, \vec{h}_2, \dots, \vec{h}_N\}$, $\vec{h}_i \in \mathbb{R}^F$, where N is the number of nodes, and F is the number of features in each node. The layer produces a new set of node features (of potentially different cardinality F'), $\mathbf{h}' = \{\vec{h}'_1, \vec{h}'_2, \dots, \vec{h}'_N\}$, $\vec{h}'_i \in \mathbb{R}^{F'}$, as its output. A *weight matrix*, $\mathbf{W} \in \mathbb{R}^{F' \times F}$, is applied to every node. The attention mechanism a is a single-layer feedforward neural network, parametrized by a weight vector $\vec{a} \in \mathbb{R}^{2F'}$, and applying the LeakyReLU nonlinearity. The coefficients computed by the attention mechanism can be expressed as [29]:

$$\alpha_{ij} = \frac{\exp \left(\text{LeakyReLU} \left(\vec{a}^T [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_j] \right) \right)}{\sum_{k \in \mathcal{N}_i} \exp \left(\text{LeakyReLU} \left(\vec{a}^T [\mathbf{W}\vec{h}_i \parallel \mathbf{W}\vec{h}_k] \right) \right)} \quad (1)$$

where superscript T represents the transposition operation and $\parallel$ is the concatenation operation. The attention coefficient α_{ij} indicates the importance of node j 's features to node i , where $\mathcal{N}_i$ is some neighborhood of node i in the graph.

The multi-head attention coefficients are used to compute a linear combination of the features corresponding to them, to serve as the final output features for every node :

$$\vec{h}_i' = \sigma \left(\frac{1}{K} \sum_{k=1}^K \sum_{j \in N_i} \alpha_{ij}^k \mathbf{W}^k \vec{h}_j \right) \quad (2)$$

where α_{ij}^k are normalized attention coefficients computed by the k -th attention mechanism (a^k), and $\mathbf{W}^k$ is the corresponding input linear transformation's weight matrix.

2.4 Motivation

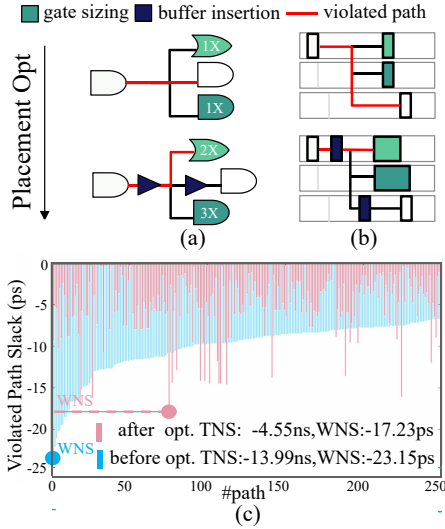


Figure 2: An example illustration of placement optimization from (a) Logical view; (b) Physical view; (c) Timing view.

Gate sizing and buffering are commonly utilized in optimization techniques during placement[24], which can have a profound influence on wirelength and interconnect parasitics as well as circuit timing. Current timing-driven placement prefers to consider immediate benefits rather than future possible outcomes, which may lead to significant re-placement overhead and even worse quality.

The influence of placement optimization could be demonstrated in Figure 2, where Figure 2 (a) and (b) provide an example for gate sizing and buffering from logical view and physical view respectively while Figure 2 (c) illustrates the corresponding path timing slack distribution from timing view before and after detail placement.

It can be found from Figure 2 (a) that for an example subcircuit with four gates, two buffers are inserted and two gates are resized after placement optimization, which induces layout changes as shown in Figure 2 (b) to fix the initial timing violation for the path marked in red. However, the optimized circuit may incur newly arisen timing violations, which indicates that the timing-driven placement may deviate from finding the optimal solution and lead to additional iterations.

As depicted in Figure 2 (c), the negative slacks of violated paths for a real design before and after optimization are compared by sorting in ascending order according to the values before optimization. After optimization, the total negative slack (TNS) is remarkably alleviated from -13.99 ns to -4.55 ns while the worst negative slack

(WNS) is improved from -23.15 ps to -17.23 ps. Although the timing slacks of most violated paths are relaxed, it could be found from Figure 2 (c) that the path slacks are not sorted in ascending order any more after optimization. For a few paths, the timing may be even worse, which reveals a disturbing fact that the procedure of placement optimization prefers to take the most critical paths into consideration while neglects the impact to the other sub-critical paths, leading to considerable risk of iteration overhead.

Motivated by the above observations, it could be concluded that the iterative process of placement optimization would be unavoidable without the comprehensive awareness of the potential physical and timing change, whose early estimation would be extremely beneficial to guide placement optimization for better quality and less iteration.

3 Proposed Placement Optimization Framework

3.1 Overview

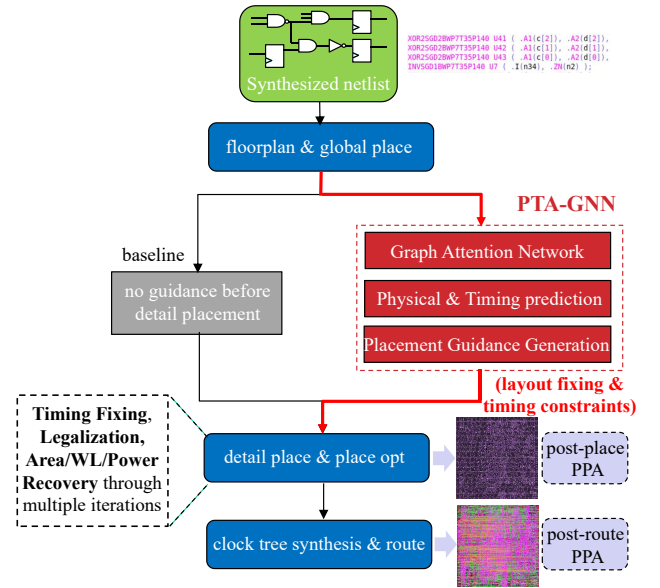


Figure 3: Default physical design flow vs. proposed PTA-GNN flow with physical and timing guidance.

Figure 3 highlights the innovations of our framework and the key differences over the native implementation of physical design flow. For a global placed netlist, different from the baseline flow without any intelligent guidance for detailed placement, our proposed **PTA-GNN** framework generates physical- and timing-related guidance for commercial tools before detail placement.

As shown in Figure 3, the synthesized netlist is designed by floorplaning and global placement, from which physical and timing related features are extracted as the input of graph attention networks. By sampling and aggregating features from its k -hop neighbors, the physical and timing results for gate sizing, buffer insertion and path slack during detail placement are predicted separately so as to directly generate an optimized netlist and report the potential timing violations after detail placement. Furthermore,

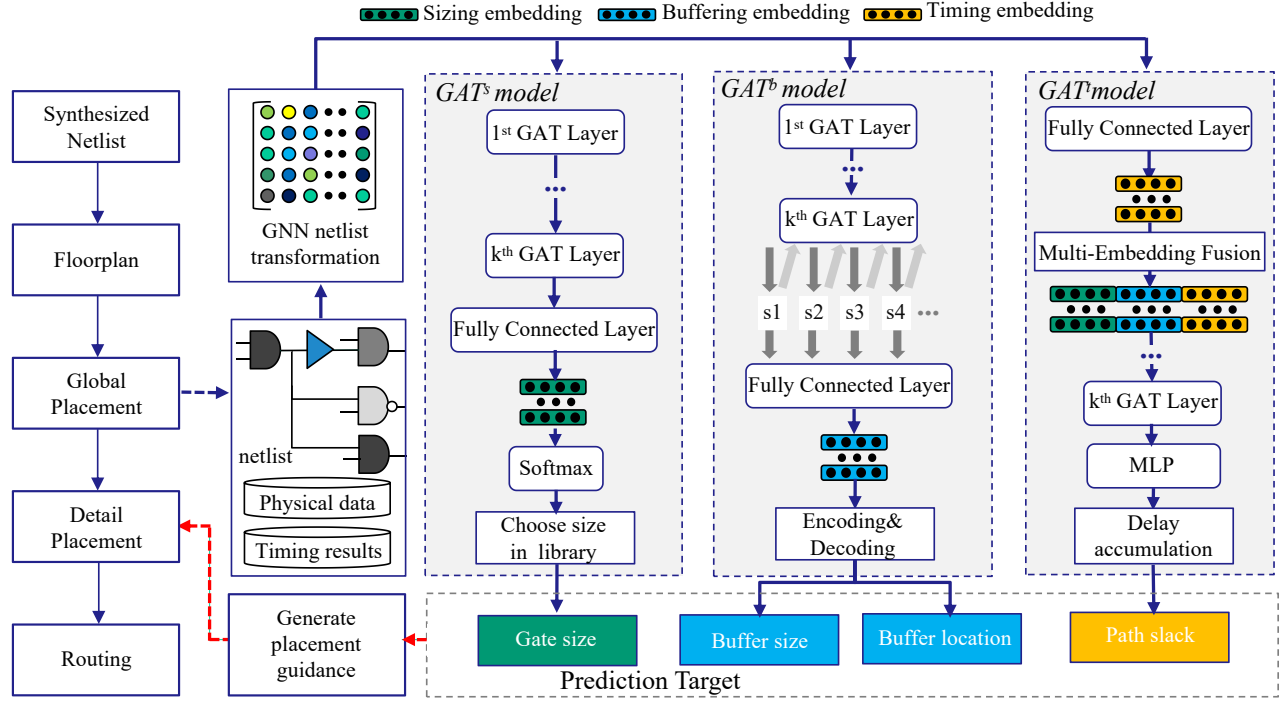


Figure 4: Illustration of our *PTA-GNN* framework, which consists of prediction and optimization for physical design flow. At the first step, *PTA-GNN* transforming netlist information in representation. Then, our models capture the relationship between the edge and cell to predict physical and timing target. Finally, *PTA-GNN* provides guidance files and commands for detail placement to improve commercial tools.

the predicted candidate optimization solutions are used as commercial tools placement guidance by transforming them into scripts for layout fixing and timing constraints. Guidance of *PTA-GNN* would be extremely beneficial for later PnR stages and tool-driven optimization including timing fixing, legalization, and etc. It facilitates faster convergence in timing closure and reduces unnecessary additional design iterations.

3.2 Feature Selection

In order to learn the physical and timing information from the circuit netlist at global placement stage, the netlist is initially converted into a directed acyclic graph (DAG) with logic cells as nodes and interconnections as edges, where the features are extracted from timing reports and design layout, as listed in Table 1.

The timing features are composed of the pin, cell arcs and Elmore delay results for the given cell after global placement while the physical features include cell function, resistance/capacitance, driving/load strength, area, coordinate, maximum Manhattan distance between the fanin cells and the given cell as well as the numbers of fanin/fanout cells for its 1-hop neighbouring cells. Rich features are necessary not only to learn the physical and timing information but also to understand the local structure, which is critical for accurate optimization-aware predictions.

3.3 Physical and Timing Prediction

Figure 4 depicts the our placement guidance generation flow with *PTA-GNN*, which provides effective physical and timing guidance

Table 1: initial features for feature matrix construction.

Type	Parameters	Description
Timing feature	r/f input slack	rise/fall slack of input pin(s)
	r/f output slack	rise/fall slack of output pin
	r/f input slew	rise/fall slew of input pin(s)
	r/f output slew	rise/fall slew of output pin
	r/f cell arcs	rise/fall delay of cell arcs
	wst cell delay	worst delay of cell
	stage delay	Elmore stage delay
Physical feature	function type	function type of cell
	tot input res	total input resistance
	tot output res	total output resistance
	downstream cap	Elmore downstream capacitance
	driving strength	driving strength of drive cell
	load strength	driving strength of load cell
	cell area	area of cell
	cell x/y coordinate	coordinate of cell
	max cell distance	max Manhattan distance
	fanins	fanin number of cell
	fanouts	fanout number of cell

to bridge the gap between global and detail placement stages. The details of each step in the flow are described below.

By extracting the physical data and timing results of logic cells in the netlist after global placement, they are represented as GNN features. The GNN netlist transformation serves as the input for physical-related prediction models of GAT^s and GAT^b , as well as

timing-related prediction model of GAT^t implemented in the proposed **PTA-GNN**. We perform the feature representation learning and build deep learning network framework in Equation (3) and (4) as a basis for different prediction models in **PTA-GNN**, where $N(v)$ denotes the neighbors of node v , W_k^{agg} and W_k^{proj} denote the aggregation and projection matrices at the k -th layer of the GNN module. *mean_pool* and *sigmoid* functions represent calculating the average of feature points in a local region and applying normalization to the output of neural network. After transformation through Equation (2), the initial features of each cell h_v^0 will be transformed into h_v^K as output of k^{th} GAT Layer in Figure 4, where K denotes the total number of layers.

$$h_{N(v)}^{k-1} = \text{mean_pool} \left(\left\{ W_k^{agg} h_u^{k-1}, \forall u \in N(v) \right\} \right), \quad (3)$$

$$h_v^k = \text{sigmoid} \left(W_k^{proj} \cdot \text{concat} \left[h_v^{k-1}, h_{N(v)}^{k-1} \right] \right), \quad (4)$$

3.3.1 gate sizing model As shown in GAT^s model of Figure 4, it provides the candidate solutions for gate size. With the comprehensive set of features, the sizing solution for each logic cell is predicted by computing the multi-head attention coefficients as Equation (1) and by graph convolution layers as Equation (3) (4). Through fully connected layer and softmax function, the tentative classification result is obtained to indicate the candidate size of each given cell for detail placement optimization, whose closest downsized value would be chosen as the candidate sizing solution.

3.3.2 buffer insertion model As depicted in Figure 4, GAT^b model applies input feature through multiple GAT layers, where the representations of cells and edges are learnt and shared for the prediction tasks of all inserted buffers in the buffer tree between a specific driver cell and multiple sink cells. Considering that the buffers inserted at the posterior stages are strongly correlated to that at the preceding stage, the buffer tree is predicted in a pipelined fashion stage by stage with the information expressed as $s1$, $s2$, $s3$, and etc, which is concatenated for the generation of buffer insertion solution. Through fully connected layer, GAT^b generates the embeddings for all stages of buffer trees for encoding&decoding module, following by activation functions and labels to generate the buffer insertion solutions including buffer size and location in circuit layout respectively.

Different from the prediction of gate sizing solution, the candidate solution of buffer insertion for detail placement optimization is significantly more complex since the solutions for multiple stages of buffers in a buffer tree, rather than a single buffer cell, should be generated. To solve this issue, a *matrixized buffer tree generation mechanism* is proposed to implement the encoding & decoding module in GAT^b model, where the buffer tree is represented as a buffer matrix as depicted in Figure 5. Each column of the matrix is corresponding to the buffer stage while each element in one column denotes the potential inserted buffer at that stage, which could be applied for the prediction tasks of buffer size as well as the horizontal and vertical coordinates of buffer location respectively by training with the corresponding labels. As illustrated in Figure 5, at encoding stage, the inserted buffer tree is represented as a $F \times S$ matrix, where F indicates the number of sink cells while S denotes the maximum stages of buffer tree. For the stage of $s1$, $s2$, $s3$ and $s4$,

different sizes of buffers or no buffer may be inserted at different rows of buffer matrix for different sink cells. At decoding stage, the matrixized representation of buffer tree is transformed back into the topological structure and the buffers in adjacent stages are connected to generate the circuit path, e.g. $p1$, $p2$, and $p3$.

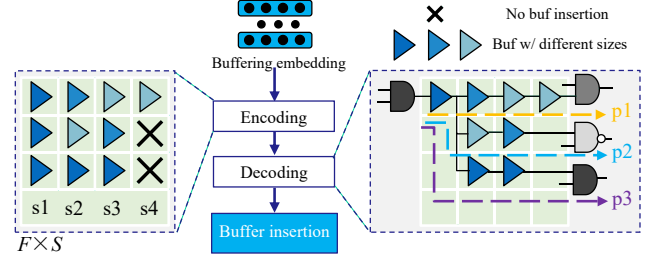


Figure 5: Illustration of matrixized buffer tree generation mechanism.

It should be noted that during encoding stage, the identical buffer shared by multiple sink cells would be represented duplicately in the matrix row for each corresponding sink cell, such as the buffers at stage $s1$ share by all three paths $p1$, $p2$ and $p3$, which may induce inaccurate buffer insertion solution and should be removed during decoding stage. To address this problem, Algorithm 1 is presented to remove the buffers predicted in adjacent rows with sufficiently small distance and identical size. Given the predefined minimum distance d_{min} according to technology file as well as the definitions for buffer size and Manhattan distance for buffers in adjacent levels in Line 1-4, the potential duplicate buffer instances are searched and moved in the loops in Line 5-11. As can be seen in Line 7-8, when the Manhattan distance for buffers in adjacent levels is less than d_{min} and their sizes are identical, the buffer in the lower level is removed for building new connections of buffer trees. In this way, the buffer tree structure is decoded from the matrixized representation in encoding stage in Line 12.

Algorithm 1: Duplicate buffer removal procedure

Input: predicted buffer size B with GAT^b , predicted buffer location $\{x, y\}$ with GAT^b , buffer matrix $F \times S$
Output: Generated solution for buffer tree insertion

- 1: **Define** d_{min} : minimum distance value
- 2: **Define** $buf_s^{i,j}$: buffer instance at stage i and level j
- 3: **Define** $buf_s^{i,j}$: buffer size at stage i and level j according to B
- 4: **Define** $M^{i,j}$: Manhattan distance between $buf_s^{i,j}$ and $buf_s^{i,j+1}$ according to $\{x, y\}$
- 5: **for** $i \leftarrow 0$ to S **do**
- 6: **for** $j \leftarrow 0$ to $F - 1$ **do**
- 7: **if** $M^{i,j} < d_{min}$ **and** $buf_s^{i,j} == buf_s^{i,j+1}$ **then**
- 8: **Remove** $\{buf_s^{i,j}\}$
- 9: **end if**
- 10: **end for**
- 11: **end for**
- 12: **return** buffer tree structure

3.3.3 timing model Due to the undoubted impact induced from gate sizing and buffer insertion, a *Multi-Embedding Fusion* technique is utilized in timing aware guidance generation by fusing the learnt embeddings from GAT^s and GAT^b models from gate sizing and buffer insertion, so as to improve prediction accuracy with physical insight.

As shown in the GAT^t model of Figure 4, after the initial timing embedding is generated with a fully connected layer, it is fused with both sizing embedding and buffering embedding, which is processed through the k aggregation layers and encoding layers of GAT with self-attention mechanism. For each original cell in netlist, its cell delay is predicted by Multi-Layer Perceptron (MLP) and then the path slack could be obtained by cell delay accumulation and timing constraint information.

3.4 Physical and Timing Aware Optimization

Utilizing prediction models to drive complex implementation flows and achieve better PPA is undoubtedly a key aspect of the design automation tools. Owing to the predicted gate sizing and buffering solution as well as path slacks in Section 3.3, the physical and timing aware guidance could be generated to improve commercial tools with better PPA metrics as illustrated in Figure 4.

In order to be integrated into commercial tools, the corresponding commands are generated as *Tcl script* according to the predicted netlist changes and timing results. In this work, *Synopsys IC Compiler* is used as placement tool and guided via the following commands.

- **size_cell** According to the predicted gate sizing solution by GAT^s , the logic cells in the netlist after global placement stage are upsized or downsized by this command.
- **insert_buffer** According to the predicted buffer insertion solution by GAT^b , the candidate buffers are inserted with the appropriate driven strengths and locations.
- **group_path** According to the predicted path slack by GAT^t , the potential violated and near-violated paths are grouped separately with customized group weights, which guides the tool to pay equivalent level of attention to the concerned paths.

4 Experiment Results

4.1 Experiment Setup

The proposed placement optimization framework based on *PTA-GNN* was implemented with Pytorch and PyG. Model training and testing were performed on a Linux machine equipped with an NVIDIA Tesla V100 GPU and two Intel Xeon CPUs at 2.20 GHz and 256 GB memory. The layer numbers and multi-head-attentions number of GAT are respectively 5 and 8 while the dimension for hidden embedding is 128. The learning rate α is 0.001 and the weight decay is 0.0001.

Thirteen real-world circuits comprising different functions from *OpenCores* were used as a benchmark set to validate the optimization framework, which were synthesized and placed by *Synopsys Design Compiler* and *Synopsys IC Compiler* respectively with TSMC 22nm process and the ground-truth timing slacks were analyzed

Here, we use timing annotation commands used in *Synopsys IC Compiler*. However, the commands can be replaced with compatible ones used in other tools such as *Cadence Innovus* or *Synopsys IC Compiler II*

with *Synopsys PrimeTime*. The standard physical design flow using aforementioned commercial tools serves as baseline, requires 7.2 hours for training benchmarks and 4.3 hours for test benchmarks to complete the total process of physical design respectively. And our *PTA-GNN* framework utilizes *Tcl* script to generate guidance for baseline before detail placement.

The benchmarks and circuit statistics details are shown on the left part of Table 2. Seven designs were used for training and the remaining six for testing. The training process for the proposed *PTA-GNN* takes about 8.3 hours totally on the GPU, while the time is comparable to the physical design time of the training circuit. In Table 2, *#cell*, *#edge*, *#EPs* and *#ori. buf* represent the number of cells, edges, endpoints and original buffers after global placement while *#sized cell* and *#added buf* represent the number of sized cells and inserted buffers during detail placement. It can be seen that for all designs, over 22.37%/15.03% cells were sized in training/testing set while over 31.13%/24.95% buffers were added on average, indicating significant optimization effort and nonnegligible inaccurate timing estimation during timing-driven placement stage.

4.2 Physical and Timing Prediction Evaluation

The right part of Table 2 demonstrates the prediction accuracy of *PTA-GNN* framework and the competitive works based on Transformer [23], RF [11], and GNN [7]. The metrics of F1-score and R^2 are used to evaluate the prediction accuracy of the classification problems, i.e. gate size and buffer size, and the regression problems, i.e. buffer location and path slack, respectively. The value closer to 1 indicates higher accuracy.

Owing to the models GAT^s , GAT^b and GAT^t for the predictions of gate sizing and buffer insertion as well as path slacks, the proposed *PTA-GNN* framework achieves excellent accuracy as can be seen from Table 2 that the F1-score of gate sizing and buffer size, and R^2 of buffer location and slack prediction results achieve 94.66%, 96.55%, 95.30% and 93.46% for seen designs in training set and 91.20%, 93.20%, 93.10% and 91.49% for unseen designs in test set on average. Compared to a few hours of runtime by commercial tools, the inference time of the proposed *PTA-GNN* framework is almost negligible to less than one second.

Compared with prior transformer-based gate sizing method, Transizer [23], which only achieves the R^2 of 85.56% for test set on average, our GAT-based gate sizing prediction model GAT^s has the ability to collect local perception and could capture graph topological information with rich physical and timing features.

Compared with RF-based model in [11], which is applied to buffer insertion with the F1-score of 85.56% and the R^2 of 82.74% for buffer size and location prediction, the larger receptive field of our buffer insertion prediction model GAT^b contributes to more local information and shows better results in test set while the competitive work in [11] are prone to over-fitting due to the features used only include that for the target cell and cannot capture local information from its surrounding neighbors.

Compared with the timing prediction model in [7], where the physical changes introduced by placement optimization are not taken into account, the R^2 of path slacks in unseen design is only 86.06% on average. Owing to the *Multi-Embedding Fusion* method and the self-attention mechanism for netlist change information

Table 2: Benchmark statistics and prediction accuracy comparison.

	benchmark	Circuit statistics						Prediction results							
		#cell	#edge	#EPs	#sized cell	#ori.buf	#added buf	cell size		buffer size		buffer location		path slack	
								ICCAD	GAT ^s	TODAES	GAT ^b	TODAES	GAT ^b	DAC	GAT ^s + GAT ^b
								'22[23]	(ours)	'23[11]	(ours)	'23[11]	(ours)	'22[7]	+GAT ^t (ours)
train	aes	8586	10595	79	1827	1261	416	95.77%	96.46%	86.94%	95.41%	71.34%	89.91%	86.11%	89.38%
	des	5832	6334	255	1442	198	105	85.23%	92.83%	84.32%	96.77%	86.53%	96.54%	89.17%	94.14%
	ecg	116714	123131	8070	26467	5921	2106	91.82%	91.15%	94.38%	94.73%	68.25%	96.91%	67.45%	92.53%
	mc	10734	10919	353	737	1536	97	88.76%	95.68%	93.32%	96.11%	91.61%	97.43%	83.13%	94.67%
	spi	4936	5630	273	283	231	148	90.46%	97.75%	94.18%	95.11%	94.65%	93.16%	93.74%	92.83%
	tv80_core	7336	8822	393	746	897	342	94.11%	93.53%	84.57%	99.32%	87.64%	97.34%	72.01%	93.61%
	vga_dma	96012	119376	16980	24459	848	177	91.13%	95.25%	93.87%	98.43%	91.95%	95.79%	86.88%	97.05%
	ave.	35736	40686	3771	7995	1556	485	91.04%	94.66%	90.23%	96.55%	84.57%	95.30%	82.64%	93.46%
test	ac97	15362	16867	1918	2564	1266	580	82.19%	93.01%	89.73%	94.03%	86.51%	94.64%	89.43%	92.54%
	ethernet	84542	95437	10122	19330	985	474	79.84%	90.26%	73.32%	91.92%	69.43%	93.97%	71.43%	88.34%
	eth	64676	79683	8943	12027	1065	289	86.08%	91.94%	91.00%	95.87%	88.64%	95.61%	92.41%	93.56%
	pci_bridge32	24168	27744	2251	3798	1828	257	89.22%	91.03%	88.59%	94.19%	78.96%	90.22%	89.21%	92.41%
	systemcdes	3378	3531	255	323	347	118	75.34%	88.41%	83.54%	93.51%	86.75%	91.37%	89.56%	90.68%
	wb_dma	73026	77991	871	1804	19827	4598	83.02%	92.52%	87.18%	89.68%	86.14%	92.79%	84.32%	91.43%
	ave.	44192	50209	4060	6641	4219	1053	82.62%	91.20%	85.56%	93.20%	82.74%	93.10%	86.06%	91.49%
					(15.03%)		(24.95%)								

used in GAT^t , our timing prediction model is highly beneficial to accuracy for path slack prediction.

4.3 Physical and Timing Aware Optimization Evaluation

In order to evaluate the quality improvement after detail placement with the proposed *PTA-GNN* framework, Table 3 compares it with the baseline solution, PL-GNN [19], and the results from ablation study. The baseline (*ICC*) presents the tool flow results with *IC Compiler* for test set. PL-GNN [19] proposes an advanced method that utilizes unsupervised learning to perform weighted *K*-means clustering to determine the placement groups as placement guidance and improve the default placement flow. The ablation studies included in Table 3 are used to evaluate the improvement by physical and timing aware guidance separately. The quality of the above-mentioned approaches is evaluated by WNS, TNS, #VP in Table 3 which represent the worst negative slack, total negative slack and the number of violation paths.

As shown in Table 3, the proposed *PTA-GNN* framework achieves an average of 54.21% and 33.60% for WNS and TNS reduction respectively and 11.37% reduction in #VP compared with the baseline, which indicates the outstanding placement quality of the proposed work due to accurate and appropriate physical and timing aware guidance. The improvement could even up to 89.09% and 55.47% for WNS and TNS reduction for *systemcdes* and large-scale design *wb_dma* and 21.07% #VP reduction for *pci_bridge32*.

The competitive approach with placement guidance, PL-GNN [19], shows an average of 25.93% WNS, 15.33% TNS, 0.29% #VP improvement in Table 3 compared with the baseline, which is extremely unremarkable compared with this work, mainly due to [19] does not address the netlist change and timing optimization problem. For the design *ac97*, the PL-GNN [19] even leads to a inferior

optimization results with 28.92% and 7.29% worse WNS and TNS respectively and 4.14% increase for #VP.

By analyzing the results of ablation studies shown in Table 3, it could be found that without any aspects of physical or timing guidance, the quality of placement optimization is vastly inferior to the proposed framework with both guidance, indicating the crucial impact to guide detail placement with physical and timing perspectives.

In order to demonstrate the impact of placement optimization guidance for further routing stage, the improved PPA quality is evaluated and compared with the baseline solution and recent work [5], as shown in Table 4. In work [5], a routability-aware placement flow is proposed based on Hierarchical Density-Based Spatial Clustering of Applications with Noise(HDBSCAN) algorithm to generate soft placement guidance, aiming to achieve better results and help to reduce routing overflow.

It can be observed that owing to the physical and timing aware guidance at detail placement stage, the proposed framework achieves significantly better optimization results with an average of 21.81% WNS, 24.93% TNS, 5.82% #VP, 2.19% wirelength and 0.27% area reduction compared with the baseline, which outperforms the work of [5] especially in the aspect of timing improvement. This is due to the fact that the placement violation is greatly reduced under our framework without hurting area or wirelength, which is conducive to routing stage and reduces violations after routing.

The runtime overhead for the proposed *PTA-GNN* framework is compared with the baseline and [5] as illustrated in Figure 6, which includes that for the whole physical design flow from floorplan stage to routing stage for all six unseen designs in test set. It can be observed in Figure 6 that different with the baseline, additional runtime is occupied for this work and [5] to generate placement guidance. In spite of this, the proposed framework achieves 22% runtime reduction owing to the mitigated iteration cost at placement

Table 3: Results comparison after detail placement on test benchmarks. The unit for WNS & TNS is *ns*, for VP means violation path, bigger *impr.* value indicates a greater degree of improvement, w/o means without, bold font means the best result in the same metric.

benchmark	baseline(ICC)			PL-GNN[19]			PTA-GNN w/o timing guidance			PTA-GNN w/o physical guidance			PTA-GNN(our full flow)		
	WNS	TNS	#VP	WNS (impr.)	TNS (impr.)	#VP (impr.)	WNS (impr.)	TNS (impr.)	#VP (impr.)	WNS (impr.)	TNS (impr.)	#VP (impr.)	WNS (impr.)	TNS (impr.)	#VP (impr.)
ac97	-0.166	-67.21	893	-0.214 (-28.92%)	-72.11 (-7.29%)	930 (-4.14%)	-0.166 (0.00%)	-66.71 (0.74%)	869 (2.69%)	-0.176 (-6.02%)	-65.81 (2.08%)	877 (1.79%)	-0.148 (10.84%)	-60.37 (10.18%)	808 (9.52%)
ethernet	-0.962	-112.31	138	-0.804 (16.42%)	-96.71 (13.89%)	123 (10.87%)	-0.815 (15.28%)	-97.51 (13.18%)	138 (0.00%)	-0.856 (11.02%)	-101.81 (9.35%)	138 (0.00%)	-0.771 (19.85%)	-91.91 (18.16%)	120 (13.04%)
eth	-1.028	-122.45	138	-0.822 (20.04%)	-100.03 (18.31%)	138 (0.00%)	-0.910 (11.48%)	-123.36 (-0.74%)	138 (0.00%)	-1.011 (1.65%)	-126.36 (-3.19%)	138 (0.00%)	-0.578 (43.77%)	-69.87 (42.94%)	133 (3.62%)
pci_bridge32	-0.175	-19.81	280	-0.133 (24.00%)	-11.66 (41.14%)	265 (5.36%)	-0.197 (-12.57%)	-24.92 (-25.80%)	476 (-70.00%)	-0.143 (18.29%)	-13.42 (32.26%)	268 (4.29%)	-0.114 (34.86%)	-11.63 (41.29%)	221 (21.07%)
systemcdes	-1.778	-5.09	32	-1.005 (43.48%)	-5.69 (-11.79%)	32 (0.00%)	-1.644 (7.54%)	-4.72 (7.27%)	32 (0.00%)	-0.201 (88.70%)	-5.82 (-14.34%)	32 (0.00%)	-0.194 (89.09%)	-2.42 (52.46%)	30 (6.25%)
wb_dma	-0.495	-87.56	243	-0.432 (12.73%)	-64.68 (26.13%)	231 (4.94%)	-0.467 (5.66%)	-83.48 (4.66%)	237 (2.47%)	-0.464 (6.26%)	-73.92 (15.58%)	233 (4.12%)	-0.303 (38.79%)	-38.99 (55.47%)	216 (11.11%)
ave.	-0.767	-69.07	287	-0.568 (25.93%)	-58.48 (15.33%)	286 (0.29%)	-0.699 (8.80%)	-66.78 (3.31%)	315 (-9.63%)	-0.480 (38.08%)	-64.52 (6.58%)	281 (2.20%)	-0.351 (54.21%)	-45.86 (33.60%)	254 (11.37%)

Table 4: Detailed comparison after routing, the unit for WL(wirelength) and area is *m* and *um*².

benchmark	baseline(ICC)					ISQED'23[5]					PTA-GNN(our full flow)				
	WNS	TNS	#VP	WL	Area	WNS (impr.)	TNS (impr.)	#VP (impr.)	WL (impr.)	Area (impr.)	WNS (impr.)	TNS (impr.)	#VP (impr.)	WL (impr.)	Area (impr.)
ac97	-0.146	-68.36	1019	0.064	7933	-0.146 (0.00%)	-72.13 (-5.51%)	1118 (-9.72%)	0.064 (0.00%)	7968 (-0.44%)	-0.126 (13.70%)	-51.42 (24.78%)	1009 (0.98%)	0.062 (3.13%)	7892 (0.52%)
ethernet	-4.142	-534.55	138	0.329	36348	-3.544 (14.44%)	-482.04 (9.82%)	138 (0.00%)	0.312 (5.17%)	35994 (0.97%)	-2.982 (28.01%)	-360.15 (32.63%)	121 (12.32%)	0.332 (-0.91%)	36468 (-0.33%)
eth	-3.831	-503.32	138	0.304	36265	-3.598 (6.08%)	-468.19 (6.97%)	138 (0.00%)	0.304 (0.00%)	36278 (-0.04%)	-3.298 (13.91%)	-419.19 (16.72%)	128 (7.25%)	0.301 (0.99%)	36129 (0.37%)
pci_bridge32	-0.173	-31.37	790	0.155	13204	-0.179 (-3.47%)	-25.44 (18.90%)	753 (4.68%)	0.151 (2.58%)	13253 (-0.38%)	-0.119 (31.21%)	-25.43 (18.94%)	741 (6.20%)	0.143 (7.74%)	13120 (0.63%)
systemcdes	-0.218	-6.57	32	0.014	1337	-0.237 (-8.72%)	-6.94 (-5.63%)	33 (-3.13%)	0.014 (0.00%)	1343 (-0.48%)	-0.196 (10.09%)	-6.14 (6.54%)	32 (0.00%)	0.014 (0.00%)	1341 (-0.34%)
wb_dma	-0.381	-49.63	254	0.231	16038	-0.314 (17.59%)	-35.68 (28.11%)	213 (16.14%)	0.229 (0.87%)	16061 (-0.14%)	-0.231 (39.37%)	-33.88 (31.73%)	202 (20.47%)	0.221 (4.33%)	15871 (1.04%)
ave.	-1.482	-198.96	395	0.182	18521	-1.336 (9.82%)	-181.73 (8.66%)	398 (-0.93%)	0.179 (2.10%)	18488 (0.18%)	-1.158 (21.81%)	-149.37 (24.93%)	372 (5.82%)	0.178 (2.19%)	18470 (0.27%)

stage, which is much better than the work of [5]. An interesting finding is that although the runtime benefit is 15% for detail placement, a considerable income is obtained at routing stage by reducing the runtime to 63% of the baseline, which is possible due to the better routability induced by the optimized placement results. We attribute the observed reduction in runtime to the proactive role played by deep learning within *PTA-GNN*. By providing superior initial solutions for netlist, *PTA-GNN* facilitates a more efficient placement process, while potential violation path guidance effectively minimizes iterations number of the placement algorithm. Furthermore, the enhanced placement outcomes are instrumental in promoting the rapid convergence of the routing without excessive iterations.

4.4 Why Does PTA-GNN Work

In the above experiments, our methods consistently demonstrated superior results compared to state-of-the-art approaches and industrial commercial tool across different benchmarks and objectives.

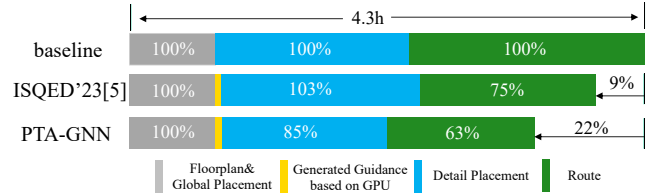


Figure 6: Illustration of runtime comparison. yellow part is mainly composed by the time for feature data collection, data processing, and inference.

We believe the achievements of our framework can be accounted by the following reasons:

4.4.1 physical aware guidance The optimal placed layout cannot be theoretically obtained because the quality of placement is heavily

dependent on the immediate PPA benefits and subsequent optimization strategy. To assist and accelerate placement optimization strategy, it is necessary to improve circuit topology quality before detail placement optimization. This problem is addressed effectively in our work by introducing optimized netlist, which relies on our physical aware guidance of gate sizing and buffer insertion solutions, to avoid the numerous iterations of netlist reconstruction as far as possible.

4.4.2 timing aware guidance Lacking timing information at early stage, placement optimization driven by timing closures can not weight data path accurately. Proposed **PTA-GNN** provides the optimized netlist and updates timing constraints to mitigate the impact of layout changes on the placed and unreplaced nets/cells, which are significantly affected by placement timing optimization. It is beneficial to specify path delay weight guidance, which can significantly reduce violation paths, resulting in more enhanced performance.

5 Conclusion

In this work, a placement optimization framework based on **PTA-GNN** was proposed to improve the optimization of timing closures across benchmarks in advanced technologies 22nm. We explore deep GNN architectures and the potential of deep learning in VLSI design flow. This work shall demonstrate the effectiveness of the proposed optimization framework. In the future, we aim to expand our framework for full-flow optimization and further exploration of different techniques to optimize design flow.

References

- [1] Erick Carvajal Barboza, Nishchal Shukla, Yiran Chen, and Jiang Hu. 2019. Machine Learning-Based Pre-Routing Timing Prediction with Reduced Pessimism. In *Proc. DAC*.
- [2] Peng Cao, Guoqing He, and Tai Yang. 2023. TF-Predictor: Transformer-Based Prerouting Path Delay Prediction Framework. *IEEE TCAD* (2023).
- [3] Wei-Ting J. Chan, Pei-Hsin Ho, Andrew B. Kahng, and Prashant Saxena. 2017. Routability Optimization for Industrial Designs at Sub-14nm Process Nodes Using Machine Learning. In *Proc. ISPD*.
- [4] Kyungjoon Chang, Jaehoon Ahn, Heechun Park, Kyu-Myung Choi, and Taewhan Kim. 2023. DTOC: integrating Deep-learning driven Timing Optimization into the state-of-the-art Commercial EDA tool. In *Proc. DATE*.
- [5] Chieh-Yu Cheng and Ting-Chi Wang. 2023. Routability-aware Placement Guidance Generation for Mixed-size Designs. In *Proc. ISQED*.
- [6] Hsien-Han Cheng, Iris Hui-Ru Jiang, and Oscar Ou. 2020. Fast and Accurate Wire Timing Estimation on Tree and Non-Tree Net Structures. In *Proc. DAC*.
- [7] Zizheng Guo, Mingjie Liu, Jiaqi Gu, Shuhan Zhang, David Z. Pan, and Yibo Lin. 2022. A timing engine inspired graph neural network model for pre-routing slack prediction. In *Proc. DAC*.
- [8] Wenkai He, Xiaqing Li, Xinkai Song, Yifan Hao, Rui Zhang, Zidong Du, and Yunji Chen. 2023. Chip design with machine learning: a survey from algorithm perspective. *Science China Information Sciences* (2023).
- [9] Xu He, Zhiyong Fu, Yao Wang, Chang Liu, and Yang Guo. 2022. Accurate Timing Prediction at Placement Stage with Look-Ahead RC Network. In *Proc. DAC*.
- [10] Andrew B. Kahng. 2021. Advancing Placement. In *Proc. ISPD*.
- [11] Suhas Krishna Kashyap and Sule Ozev. 2023. IMPROVED: Integrated Method to Predict PostRouting Setup Violations in Early Design Stages. *TODAES* (2023).
- [12] W.N. Li. 1993. Strongly NP-hard discrete gate sizing problems. In *Proc. IEEE ICCD*.
- [13] Rongjian Liang, Anthony Agnesina, Geraldo Pradipta, Vidya A. Chhabria, and Haoxing Ren. 2023. Invited Paper: CircuitOps: An ML Infrastructure Enabling Generative AI for VLSI Circuit Optimization. In *Proc. ICCAD*.
- [14] Rongjian Liang, Siddhartha Nath, Anand Rajaram, Jiang Hu, and Haoxing Ren. 2023. BufferFormer: A Generative ML Framework for Scalable Buffering. In *Proc. ASP-DAC*.
- [15] Peiyu Liao, Dawei Guo, Zizheng Guo, Siting Liu, Yibo Lin, and Bei Yu. 2023. DREAMPlace 4.0: Timing-Driven Placement With Momentum-Based Net Weighting and Lagrangian-Based Refinement. *IEEE TCAD* (2023).
- [16] Yi-Chen Lu, Wei-Ting Chan, Deyuan Guo, Sudipto Kundu, Vishal Khandelwal, and Sung Kyu Lim. 2023. RL-CCD: Concurrent Clock and Data Optimization using Attention-Based Self-Supervised Reinforcement Learning. In *Proc. DAC*.
- [17] Yi-Chen Lu, Siddhartha Nath, Vishal Khandelwal, and Sung Kyu Lim. 2021. RL-Sizer: VLSI Gate Sizing for Timing Optimization using Deep Reinforcement Learning. In *Proc. DAC*.
- [18] Yi-Chen Lu, Siddhartha Nath, Sai Surya Kiran Pentapati, and Sung Kyu Lim. 2020. A Fast Learning-Driven Signoff Power Optimization Framework. In *Proc. ICCAD*.
- [19] Yi-Chen Lu, Sai Pentapati, and Sung Kyu Lim. 2021. The Law of Attraction: Affinity-Aware Placement Optimization Using Graph Neural Networks. In *Proc. ISPD*.
- [20] Yi-Chen Lu, Tian Yang, Sung Kyu Lim, and Haoxing Ren. 2022. Placement Optimization via PPA-Directed Graph Clustering. In *Proc. MLCAD*.
- [21] Robert Kirby et al. Mingjie Liu, Teodor-Dumitru Ene. 2023. ChipNeMo: Domain-Adapted LLMs for Chip Design. *arXiv e-prints* (2023), arXiv:2311.00176.
- [22] Siddhartha Nath and Vishal Khandelwal. 2021. Machine Learning-Enabled High-Frequency Low-Power Digital Design Implementation At Advanced Process Nodes. In *Proc. ISPD*.
- [23] Siddhartha Nath, Geraldo Pradipta, Corey Hu, Tian Yang, Bruce Khailany, and Haoxing Ren. 2022. TransSizer: A Novel Transformer-Based Fast Gate Sizer. In *Proc. ICCAD*.
- [24] Walter Lau Neto, Matheus Trevisan Moreira, Luca Amaru, Cunxi Yu, and Pierre-Emmanuel Gaillardon. 2021. Read your Circuit: Leveraging Word Embedding to Guide Logic Optimization. In *Proc. ASP-DAC*.
- [25] Seonghyeon Park, Daeyeon Kim, Seongbin Kwon, and Seokhyeong Kang. 2023. Routability Prediction and Optimization Using Explainable AI. In *Proc. ICCAD*.
- [26] Phuoc Pham and Jaeyong Chung. 2023. AGD: A Learning-based Optimization Framework for EDA and its Application to Gate Sizing. In *Proc. DAC*.
- [27] Weiping Shi, Zhuo Li, and C.J. Alpert. 2004. Complexity analysis and speedup techniques for optimal buffer insertion with minimum cost. In *Proc. ASP-DAC*.
- [28] Synopsys. 2011. IC Compiler Implementation User Guide. <https://www.synopsys.com/implementation-and-signoff/physical-implementation/ic-compiler.html>.
- [29] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Liò, and Yoshua Bengio. 2018. Graph Attention Networks.
- [30] Ziyi Wang, Siting Liu, Yuan Pu, Song Chen, Tsung-Yi Ho, and Bei Yu. 2023. Restructure-Tolerant Timing Prediction via Multimodal Fusion. In *Proc. DAC*.
- [31] Zhiyao Xie, Rongjian Liang, Xiaoqing Xu, Jiang Hu, Chen-Chia Chang, Jingyu Pan, and Yiran Chen. 2022. Preplacement Net Length and Timing Estimation by Customized Graph Neural Network. *IEEE TCAD* (2022).